My Courses

WY LAB4-1 AhoCorasick

这是一道关于Aho-Corasick Automaton（中文通常简称为AC自动机）的题目。通过完成本题，你将学习和实现一种用于“多模版字符串匹配”的算法。

简单起见，本题所说的“字符串”只包括26个小写英文字母。

评分依据实验报告为主，4个部分各占四分之一分数。

1. 用“自动机”的数学模型理解KMP算法

“自动机” (automaton) 是一种描述状态变换的数学模型。对于一个自动机，有多种方法进行描述，例如文字描述、画图描述、列表描述等。

例如，可以用一个自动机来描述“灯泡亮暗受开关影响”的状态变换。

文字描述：灯泡有亮和暗两种状态。每次按下开关，都将在亮暗之间切换状态。

画图描述:

```
     ------按开关------>
[亮]                    [暗]
     <-----按开关-------
```

我们可以用自动机的模型来重新描述一些字符串匹配的算法。

例如，对于未改进的KMP算法来说，一个长度为M的模板串，构建一个未改进的next表，其实就构建了一个M+1个状态的自动机。

这M+1个状态，分别对应：当前已经匹配了前0、1、2....M个字符。

比如，一个最简单的模板串 “aaa”，它就有4个状态，表示在当前匹配位置，能匹配0/1/2/3个连续的a，我们记作状态0/1/2/3。

它的改进版next表，在匹配失败后，就需要从头开始再匹配。

状态0/1/2/3的转移，取决于所查看的下一个字符。如果下一个字符不是a，那么转移到状态0。如果下一个字符是a，那么依次转移到下一个状态，

也就是，当遇到字符a的时候，状态0转移到状态1，状态1转移到状态2，状态2转移到状态3，状态3还是转移到状态3（从一个完美匹配的位置转移到下一个完美匹配的位置）。

这里的“状态转移”，表示的是，每当我们从上一个比较位置，移动到下一个相邻的比较位置时，给出下一个字符，当前的“匹配状态”将如何变化。

如果要画成一张图，可以依次画四个圈，里面依次标上0、1、2、3，然后0到1连一个箭头，上面写"a", 1到2、2到3、3到3也都有这样的箭头上面标记“a”。

另外，0到0、1到0、2到0、3到0，也各自有一个箭头，箭头上面标记“非a”

任务1: 用上面所说的方式，画出“aaa”的改进版next表对应的自动机。

任务2: 用上面所说的方式，画出“ababac”的未改进next表对应的自动机。

任务3: 用上面所说的方式，画出“ababac”的改进版next表对应的自动机。

在实验报告中，放三张图就可以。推荐使用docx或pdf格式提交。当然，也可以试试用ASCII Art完成画图，提交一份txt或md文件。

2. 用字典树加速多模板字符串的蛮力匹配算法

多模板字符串匹配，指的是我们有多个模板串，希望在文本串中找出每一个模板串的出现次数和位置。

一个直接的方法，就是对每一个模板串，都做一次KMP算法。但这在模板串较多的情况下不合理。

下面我们介绍一种叫做“字典树(trie)”的数据结构，可以用来加速多模板串的蛮力匹配，使得它和单模板串的蛮力匹配性能接近。

建立这棵树的方式，是将多个字符串的公共前缀合并。

例如，包含“abc” “abe” “ac”这三个字符串的一棵字典树，是这个样子的：

根节点1表示空字符串，1有一个子节点2，1连向2的边代表“a”，因此2代表第一个字符为“a”的字符串。

节点2有两个子节点3和4，2连向3的边代表“b”，2连向4的边代表“c”。因此，节点4代表“ac”这个字符串。

节点3有两个子节点5和6，3连向5的边代表“c”，3连向6的边代表“e”，因此，节点5代表“abc”，节点6代表“abe”。

任务4: 用上面的方式，画出包含“abc” “abe” “ac” “ab”这四个字符串的一棵字典树。

注意，如果节点对应给出的某个字符串，它要画得和其他节点不同（做个标记）。

实验报告中放一张图就可以。

任务5: 实现通过字典树优化多模板串匹配的蛮力算法并和KMP对比测试，要求：

花费O(模板串总长)的时间建立字典树，然后花费O(单个模板串的最大长度*文本串长度)的时间完成蛮力匹配，获得每个模板串的出现次数。称作算法A。

随后，实现“对每个模板串进行一次KMP算法”的多模板串匹配算法。称作算法B。

随后，构建两个测例，一个测例让算法A比算法B更快，一个测例让算法B比算法A更快。给出测例构造思路和测试结果。

实验报告中，包含算法A、B的代码、两个测例的构建思路和测试结果。

3. 用AC自动机实现多模板字符串匹配算法

在字典树的基础上，构建AC自动机，可以进一步加速多模板字符串匹配。可以说，它整合了上面的算法A和算法B的思路。

对于算法A，将字典树在上一个比较位置和文本串比对的结果，是得到在字典树上能匹配到的深度最大的节点，可以把这个节点称作这个位置的匹配状态。

那么，能否利用在上一个位置比较之后的匹配状态，快速得到下一个“有用”的匹配位置？理论上，从上一个位置的匹配状态，再加上一个新字符，可以唯一确定一个新的状态。

具体来说，对字典树上的每个节点，除了指向孩子的边（表示在当前匹配状态下，新增一个符合字典树结构的字符，得到一个匹配更长的状态），我们增加更多的边，表示新增一个不符合字典树结构的字符，能转移到哪一个“在字典树上匹配得尽量长”的状态（对应在字符串匹配过程中，遇到不符合字典树结构的字符）。这些“更多的边”也叫fail指针、失配指针。

如果从原先最长匹配了5个字符的状态，新增一个字符，跳转到一个最长匹配3个字符的状态，那就说明比较的起始位置连续向右移动了3个字符。

这样一个对所有字符都有状态转移边的“图”，就是AC自动机。

接下来，你需要：查找资料，**自学了解**如何构建AC自动机、如何使用AC自动机进行多模板串匹配。

你可以查询书籍、论文等资料，也可以通过搜索引擎查找wiki、博客等资料，也可以通过各种人工智能工具帮助自学。

任务6: 自学，编写程序，实现构建AC自动机、通过AC自动机优化的多模板字符串匹配算法。并将其和之前实现的算法A、B对拍以检验算法正确性。

实验报告中，详细列出你自学的过程，查找了哪些资料、使用了哪些工具（实验报告这一部分如果缺失，会扣分，但具体用什么自学方式、用什么工具，不影响得分）。

给出你编写的算法代码（可参考网络代码，这一部分是否参考网络代码不影响得分）和对拍程序。

4. 比较AC自动机和其他的多模板字符串匹配算法

现在有这样三种多模板串的字符串匹配算法：

对每个模板串单独进行一次KMP算法

借助trie树进行蛮力的匹配。

借助AC自动机进行匹配。

任务7: 请构造2到3个测例，比较这三种算法的性能，说明何种测例最能显示AC自动机相对其他两种算法的高效，何种测例使得这三种算法拉不开差距。

实验报告中，给出测例构造思路和测试结果。

任务8: 结合你在任务6的代码，对于字符集很大的情况（例如中文），构建AC自动机需要注意什么？和字符集很小（26个字母）的情况有什么不同？

实验报告中，能讲清楚其中一点即可。

UI powered by Twitter Bootstrap (http://getbootstrap.com/).
Tsinghua Online Judge is designed and coded by Li Ruizhe.
For all suggestions and bug reports, contact oj[at]liruizhe[dot]org.